PLM Security and IP Protection

Access Control and Secure Data Sharing: Software Code and Circuit Diagram

Sample project - Apex Motion Systems PLM

1. Project Overview

Item	Details
Project name	Secure PLM: Access Control and IP Protection System
Goal	Protect design data (CAD, BOMs, specifications, patents) in a PLM system from unauthorized access, leakage and tampering, both digitally and physically
Two parts	**Software:** Python module (RBAC, clearance levels, encryption, audit log, secure share links) **Hardware:** ESP32 RFID door controller for the PLM server room / R&D lab
Tools	Python 3 with the *cryptography* library; Arduino IDE with ESP32 board package and MFRC522 library
Status	Python code tested (output in Section 5). Firmware and wiring follow standard ESP32 + MFRC522 practice; verify on your own hardware before deployment.

2. Problem and Objectives

• Product designs are a company's most valuable IP, yet PLM data is shared with suppliers, partners and many internal teams.

• Typical leaks: over-broad permissions, shared accounts, unencrypted files, emailed CAD, ex-employees, unlogged access, unlocked server rooms.

• **Objective 1:** only the right person, in the right project, with the right clearance can open a document.

• **Objective 2:** data stays encrypted at rest and is shared only through expiring, recipient-bound links.

• **Objective 3:** every access attempt is logged in a tamper-evident audit trail.

• **Objective 4:** only authorized staff can physically reach the servers and prototype lab.

3. Threat Model and Controls

Threat	Example	Control in this project
Unauthorized internal access	Engineer opens another project's design	Need-to-know (project membership) check
Privilege abuse	Supplier downloads internal CAD	Role permissions + clearance level + 'supplier visible' flag
Data theft from storage	Stolen disk or backup	AES-256-GCM per-document encryption
Uncontrolled sharing	Link forwarded to a competitor	HMAC-signed link, expiry time, bound to one recipient
Insider cover-up	Admin edits the log	SHA-256 hash-chained audit log detects edits
Leak attribution	Screenshot or copy of a file	Visible watermark with user name and level
Physical intrusion	Walk-in to server room or lab	RFID door lock, lockout after 3 bad cards, event log

4. System Architecture

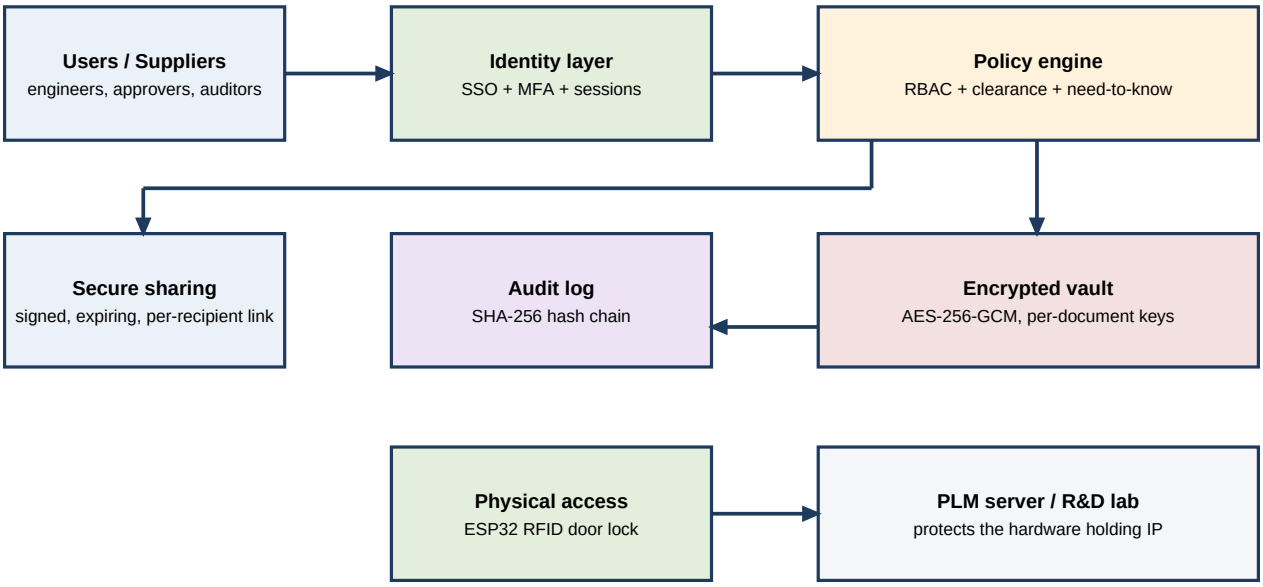


Figure 0: Logical architecture. Requests pass the identity layer, then the policy engine, which decides access to the encrypted vault and writes every decision to the audit log.

Access policy matrix

Role	Permissions	Max clearance
admin	read, write, delete, share, approve	SECRET (3)
engineer	read, write, share	CONFIDENTIAL (2)
approver	read, approve	CONFIDENTIAL (2)
auditor	read	CONFIDENTIAL (2)
supplier	read (only documents released to suppliers)	INTERNAL (1)

Rules applied in order: (1) role holds the permission, (2) clearance is at least the document level, (3) user belongs to the document's project (admin exempt), (4) suppliers only see released documents, (5) SECRET documents can never be shared externally.

5. Software Code (Python)

Save as **plm_security.py**. Install the dependency with `pip install cryptography`, then run `python plm_security.py`.

```
"""plm_security.py - Access control, encryption, audit and secure sharing for PLM data."""
import json, time, hmac, hashlib, base64, os
from dataclasses import dataclass, field
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes

LEVELS = {"PUBLIC": 0, "INTERNAL": 1, "CONFIDENTIAL": 2, "SECRET": 3}
ROLE_PERMS = {
    "admin":    {"read", "write", "delete", "share", "approve"},
    "engineer": {"read", "write", "share"},
    "approver": {"read", "approve"},
    "supplier": {"read"},
    "auditor":  {"read"},
}
ROLE_CLEARANCE = {"admin": 3, "engineer": 2, "approver": 2, "supplier": 1, "auditor": 2}

@dataclass
class User:
    name: str
    role: str
    projects: set = field(default_factory=set)

@dataclass
class Document:
    doc_id: str
    title: str
    project: str
    level: str
    supplier_visible: bool = False

def check_access(user, action, doc):
    """RBAC (role permission) + clearance level + need-to-know (project) rules."""
    if action not in ROLE_PERMS.get(user.role, set()):
        return False, f"role '{user.role}' lacks '{action}' permission"
    if ROLE_CLEARANCE[user.role] < LEVELS[doc.level]:
        return False, f"clearance too low for {doc.level}"
    if user.role != "admin" and doc.project not in user.projects:
        return False, "not a member of project (need-to-know)"
    if user.role == "supplier" and not doc.supplier_visible:
        return False, "document not released to suppliers"
    if action == "share" and doc.level == "SECRET":
        return False, "SECRET documents cannot be shared externally"
    return True, "allowed"

class AuditLog:
    """Tamper-evident log: every entry stores the hash of the previous one."""
    def __init__(self):
        self.entries = []
    def add(self, user, action, doc_id, allowed, reason):
        prev = self.entries[-1]["hash"] if self.entries else "0" * 64
        e = {"ts": time.strftime("%Y-%m-%d %H:%M:%S"), "user": user, "action": action,
             "doc": doc_id, "allowed": allowed, "reason": reason, "prev": prev}
        e["hash"] = hashlib.sha256(json.dumps(e, sort_keys=True).encode()).hexdigest()
        self.entries.append(e)
    def verify(self):
        prev = "0" * 64
        for e in self.entries:
            body = {k: v for k, v in e.items() if k != "hash"}
            ok = hashlib.sha256(json.dumps(body, sort_keys=True).encode()).hexdigest()
            if e["prev"] != prev or e["hash"] != ok:
                return False
            prev = e["hash"]
        return True

class Vault:
    """AES-256-GCM encryption at rest, one derived key per document."""
    def __init__(self, master_key=None):
        self.master = master_key or os.urandom(32)
        self.store = {}
    def _key(self, doc_id):
        return HKDF(hashes.SHA256(), 32, None, doc_id.encode()).derive(self.master)
    def put(self, doc_id, data: bytes):
        nonce = os.urandom(12)
        self.store[doc_id] = nonce + AESGCM(self._key(doc_id)).encrypt(nonce, data, doc_id.encode())
    def get(self, doc_id) -> bytes:
        blob = self.store[doc_id]
        return AESGCM(self._key(doc_id)).decrypt(blob[:12], blob[12:], doc_id.encode())

def _b64(b): return base64.urlsafe_b64encode(b).decode().rstrip("=")
```

```
def _unb64(s): return base64.urlsafe_b64decode(s + "=" * (-len(s) % 4))

class SecurePLM:
    def __init__(self):
        self.vault, self.log, self.docs = Vault(), AuditLog(), {}
        self.sign_key = os.urandom(32)
    def add_document(self, doc, data):
        self.docs[doc.doc_id] = doc
        self.vault.put(doc.doc_id, data)
    def _gate(self, user, action, doc_id):
        ok, why = check_access(user, action, self.docs[doc_id])
        self.log.add(user.name, action, doc_id, ok, why)
        if not ok:
            raise PermissionError(why)
    def read(self, user, doc_id):
        self._gate(user, "read", doc_id)
        mark = f"[{self.docs[doc_id].level} | licensed to {user.name}]\n".encode()
        return mark + self.vault.get(doc_id)        # visible watermark for traceability
    def share_link(self, user, doc_id, recipient, ttl_s=3600):
        self._gate(user, "share", doc_id)
        body = _b64(json.dumps({"d": doc_id, "to": recipient, "exp": time.time() + ttl_s}).encode())
        sig = _b64(hmac.new(self.sign_key, body.encode(), hashlib.sha256).digest())
        return f"{body}.{sig}"
    def open_link(self, token, recipient):
        body, sig = token.split(".")
        good = _b64(hmac.new(self.sign_key, body.encode(), hashlib.sha256).digest())
        if not hmac.compare_digest(sig, good):
            raise PermissionError("invalid signature")
        p = json.loads(_unb64(body))
        if p["to"] != recipient or time.time() > p["exp"]:
            self.log.add(recipient, "open_link", p["d"], False, "wrong recipient or expired")
            raise PermissionError("link expired or not issued to you")
        self.log.add(recipient, "open_link", p["d"], True, "valid link")
        return f"[SHARED | {recipient}]\n".encode() + self.vault.get(p["d"])

if __name__ == "__main__":
    plm = SecurePLM()
    plm.add_document(Document("D-100", "Gearbox CAD", "Apex-GB", "CONFIDENTIAL"), b"CAD DATA v3")
    plm.add_document(Document("D-200", "Motor IP secret", "Apex-GB", "SECRET"), b"PATENT DRAFT")
    plm.add_document(Document("D-300", "Bracket drawing", "Apex-GB", "INTERNAL", True), b"DRAWING")

    alice = User("alice", "engineer", {"Apex-GB"})
    sam = User("sam", "supplier", {"Apex-GB"})
    eve = User("eve", "engineer", {"Other"})
    root = User("root", "admin")

    def attempt(label, fn):
        try:
            fn(); print(f"[ OK ] {label}")
        except PermissionError as e:
            print(f"[DENY] {label} -> {e}")

    attempt("alice reads D-100",              lambda: plm.read(alice, "D-100"))
    attempt("alice reads D-200 (SECRET)",     lambda: plm.read(alice, "D-200"))
    attempt("eve reads D-100 (other project)", lambda: plm.read(eve, "D-100"))
    attempt("sam reads D-100 (CONFIDENTIAL)", lambda: plm.read(sam, "D-100"))
    attempt("sam reads D-300 (released)",     lambda: plm.read(sam, "D-300"))
    attempt("root shares D-200 (SECRET)",     lambda: plm.share_link(root, "D-200", "sam"))
    tok = plm.share_link(alice, "D-300", "sam", ttl_s=60)
    attempt("sam opens link",                 lambda: plm.open_link(tok, "sam"))
    attempt("eve opens sam's link",           lambda: plm.open_link(tok, "eve"))
    print("Audit chain valid:", plm.log.verify())
    plm.log.entries[1]["allowed"] = True        # attacker edits the log
    print("After tampering  :", plm.log.verify())
```

Sample output (actual run)

```
[ OK ] alice reads D-100
[DENY] alice reads D-200 (SECRET) -> clearance too low for SECRET
[DENY] eve reads D-100 (other project) -> not a member of project (need-to-know)
[DENY] sam reads D-100 (CONFIDENTIAL) -> clearance too low for CONFIDENTIAL
[ OK ] sam reads D-300 (released)
[DENY] root shares D-200 (SECRET) -> SECRET documents cannot be shared externally
[ OK ] sam opens link
[DENY] eve opens sam's link -> link expired or not issued to you
Audit chain valid: True
After tampering  : False
```

How it works

• **check_access()** is the single decision point; every read and share goes through it and is logged, allowed or denied.

• **Vault** derives a separate AES-256 key per document with HKDF and binds the ciphertext to the document ID (AAD), so blobs cannot be swapped.

• **AuditLog** chains entries by hash; changing any past entry makes *verify()* return False (shown above).

• **Share links** contain document, recipient and expiry, signed with HMAC-SHA256; a forwarded link fails for anyone else.

6. Hardware: ESP32 RFID Door Lock

Physical security for the room that holds the PLM servers or the prototype lab. The ESP32 reads an RFID card, compares its UID with an authorized list, and energizes a relay that releases a 12 V solenoid lock for 5 seconds.

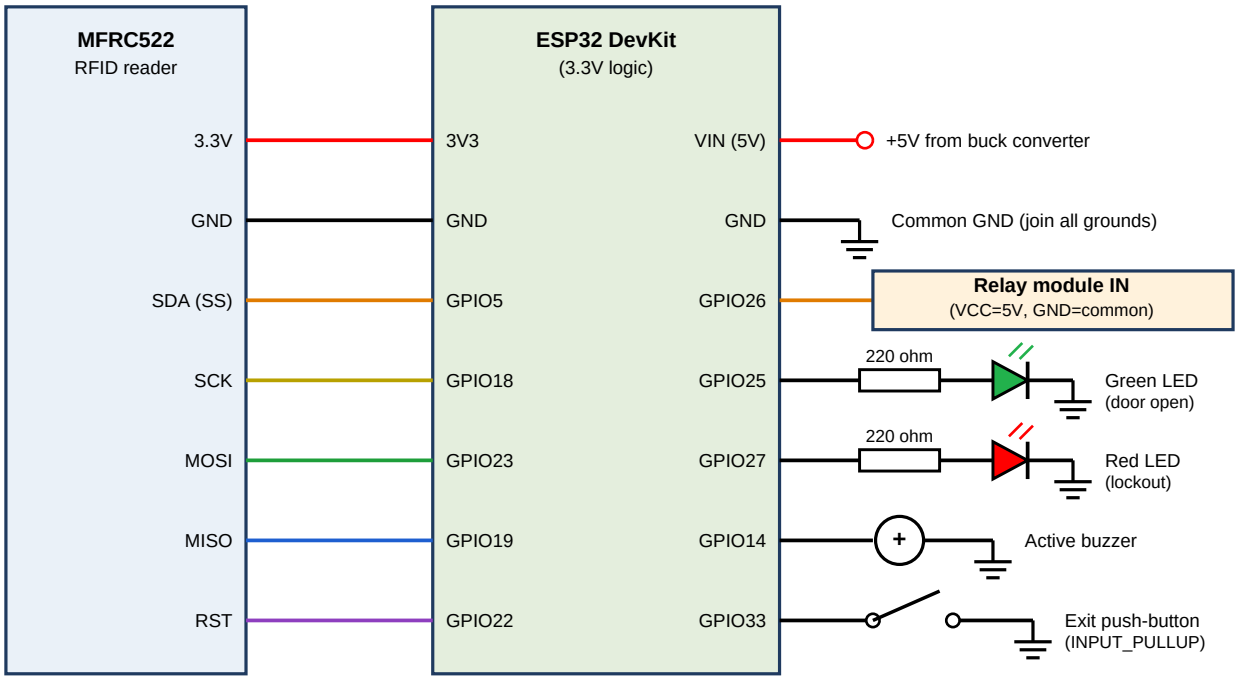
6.1 Components

Qty	Component	Notes
1	ESP32 DevKit (30-pin)	Main controller, 3.3 V logic
1	MFRC522 RFID module + tags/cards	13.56 MHz, SPI, 3.3 V only
1	5 V single-channel relay module	Opto-isolated; check if active-HIGH or active-LOW
1	12 V solenoid door lock	About 0.5 A; fail-secure (locked when unpowered)
1	12 V 2 A DC adapter	Powers lock and buck converter
1	LM2596 buck converter	Set output to 5.0 V before connecting the ESP32
1	1N4007 diode	Flyback protection across the solenoid
2 + 2	Green and red LEDs, 220 ohm resistors	Status indicators with current limiting
1	Active buzzer (3.3-5 V)	If it draws over 12 mA, drive it through a 2N2222 transistor with a 1 k ohm base resistor
1	Push button, 1 A fuse + holder, wires, breadboard	Exit button, protection and wiring

6.2 Pin connections

From	To	Function
MFRC522 3.3V	ESP32 3V3	Power (never 5 V)
MFRC522 GND	ESP32 GND	Ground
MFRC522 SDA (SS)	ESP32 GPIO5	SPI chip select
MFRC522 SCK	ESP32 GPIO18	SPI clock
MFRC522 MOSI	ESP32 GPIO23	SPI data to reader
MFRC522 MISO	ESP32 GPIO19	SPI data from reader
MFRC522 RST	ESP32 GPIO22	Reset
Relay IN	ESP32 GPIO26	Lock control
Relay VCC / GND	5 V buck output / GND	Relay coil supply
Green LED anode via 220 ohm	ESP32 GPIO25	Door open indicator (cathode to GND)
Red LED anode via 220 ohm	ESP32 GPIO27	Lockout indicator (cathode to GND)
Buzzer +	ESP32 GPIO14	Audible feedback (other side to GND)
Exit button	ESP32 GPIO33 and GND	Internal pull-up, pressed = LOW
Buck 5 V output	ESP32 VIN	Board power

6.3 Circuit diagrams

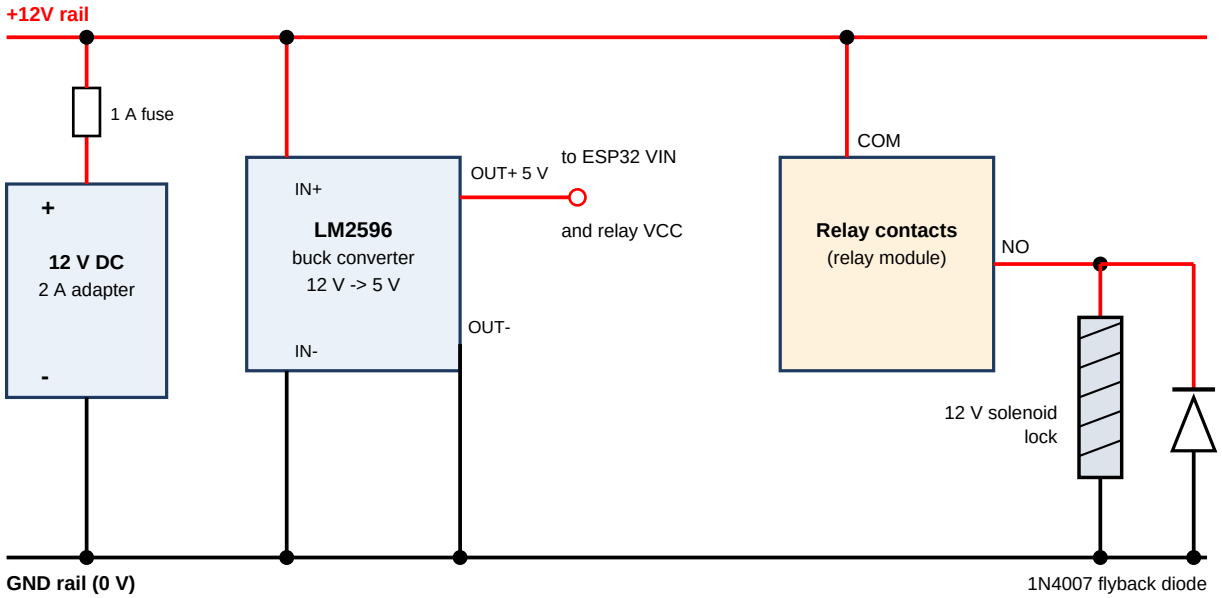


MFRC522 is a 3.3 V device: connect its supply to 3V3 only, never to 5V.

ESP32 SPI default (VSPI): SCK=18, MISO=19, MOSI=23, SS=5.

Relay module and ESP32 VIN are fed from the 5 V buck output (Figure 2); share GND.

Figure 1: Control wiring - RFID reader, ESP32, relay input, indicators, buzzer and exit button.



Relay coil is switched by ESP32 GPIO26 through the relay module IN pin (Figure 1).

Diode bar (cathode) faces +12 V; it absorbs the voltage spike when the solenoid de-energizes.

Figure 2: Power and lock circuit - 12 V supply, buck converter, relay contacts, solenoid and flyback diode.

Safety notes

• Set the buck converter to 5.0 V with a multimeter **before** connecting the ESP32 and relay module.

• Keep the 12 V lock wiring away from the SPI lines; share one common ground between 12 V, 5 V and ESP32.

• Never remove the flyback diode: the solenoid spike can reset the ESP32 or damage the relay.

6.4 Firmware (Arduino C++ for ESP32)

Save as **esp32_door.ino**. In Arduino IDE install the ESP32 board package and the library *MFRC522* (by GithubCommunity). Read your own card UIDs from the Serial Monitor (115200 baud) and replace the sample UIDs in `AUTH_UIDS`.

```
// PLM server-room / R&D lab RFID door controller (ESP32 + MFRC522 + relay + solenoid lock)
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN     5
#define RST_PIN    22
#define RELAY_PIN  26      // relay module IN (active HIGH)
#define GREEN_LED  25
#define RED_LED    27
#define BUZZER     14
#define EXIT_BTN   33      // to GND, internal pull-up

const byte AUTH_UIDS[][4] = { {0xDE,0xAD,0xBE,0xEF}, {0x12,0x34,0x56,0x78} };  // replace
const int  N_AUTH      = sizeof(AUTH_UIDS) / sizeof(AUTH_UIDS[0]);
const unsigned long UNLOCK_MS  = 5000;    // door stays unlocked 5 s
const unsigned long LOCKOUT_MS = 30000;   // lockout after 3 bad cards
const int  MAX_FAILS   = 3;

MFRC522 rfid(SS_PIN, RST_PIN);
unsigned long unlockUntil = 0, lockoutUntil = 0;
int fails = 0;

void beep(int n, int ms) {
  for (int i = 0; i < n; i++) { digitalWrite(BUZZER, HIGH); delay(ms);
                                digitalWrite(BUZZER, LOW);  delay(ms); }
}

bool isAuthorized() {
  if (rfid.uid.size != 4) return false;
  for (int i = 0; i < N_AUTH; i++) {
    bool match = true;
    for (int b = 0; b < 4; b++) if (rfid.uid.uidByte[b] != AUTH_UIDS[i][b]) match = false;
    if (match) return true;
  }
  return false;
}

void unlockDoor(const char* why) {
  digitalWrite(RELAY_PIN, HIGH);
  digitalWrite(GREEN_LED, HIGH);
  unlockUntil = millis() + UNLOCK_MS;
  Serial.printf("[%lu] UNLOCK (%s)\n", millis(), why);
  beep(1, 100);
}

void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT); pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);   pinMode(BUZZER, OUTPUT);
  pinMode(EXIT_BTN, INPUT_PULLUP);
  digitalWrite(RELAY_PIN, LOW);                 // locked by default
  SPI.begin(18, 19, 23, SS_PIN);                // SCK, MISO, MOSI, SS
  rfid.PCD_Init();
  Serial.println("Door controller ready");
}

void loop() {
  unsigned long now = millis();

  if (unlockUntil && now > unlockUntil) {       // auto re-lock
    digitalWrite(RELAY_PIN, LOW); digitalWrite(GREEN_LED, LOW); unlockUntil = 0;
  }
  digitalWrite(RED_LED, now < lockoutUntil);    // red = lockout active

  if (digitalRead(EXIT_BTN) == LOW) unlockDoor("exit button");
```

```
  if (now < lockoutUntil) return;               // ignore cards during lockout
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return;

  if (isAuthorized()) {
    fails = 0;
    unlockDoor("authorized card");
  } else {
    fails++;
    Serial.printf("[%lu] DENIED (fail %d/%d)\n", now, fails, MAX_FAILS);
    beep(3, 80);
    if (fails >= MAX_FAILS) { lockoutUntil = now + LOCKOUT_MS; fails = 0;
                              Serial.println("LOCKOUT started"); }
  }
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
}
```

7. Testing Plan

Test	Expected result
Authorized user reads own document	Allowed, watermarked copy returned, entry logged
Engineer opens SECRET document	Denied (clearance), entry logged
User from another project opens document	Denied (need-to-know)
Supplier opens unreleased document	Denied; released document allowed
Share link opened by another person / after expiry	Denied and logged
Edit an old audit entry	verify() returns False
Authorized card at the door	Relay on for 5 s, green LED, one short beep
Unknown card	Three beeps, door stays locked
Three unknown cards in a row	30 s lockout, red LED on, cards ignored
Power cut	Lock stays locked (fail-secure solenoid)

8. Limitations and Improvements

• Plain RFID UIDs can be cloned. For real deployments use MIFARE DESFire or a card plus PIN/fingerprint, and verify cards on a server.

• The Python demo keeps keys in memory. Production should use a KMS or HSM, key rotation, and persistent storage.

• Ship audit logs to write-once storage and alert on repeated denials.

• Map these roles and levels to your PLM's native ACLs (Teamcenter, Windchill, 3DEXPERIENCE) and your identity provider.

• Add DLP controls (export restrictions, download limits, dynamic watermarking on rendered views).

• Send door events from the ESP32 to the central audit log over MQTT or HTTPS with TLS.

9. Expected Results

• Role, clearance and project checks applied consistently to every document access.

• Encrypted data at rest and time-limited, traceable external sharing.

• A tamper-evident record for audits (ISO 27001, TISAX, export-control reviews).

• Controlled physical access to the systems that hold product IP.